Introducción a React

Fundamentos para el desarrollo de aplicaciones Ionic React con Supabase

1. ¿Qué es React?

React es una biblioteca JavaScript desarrollada originalmente por Facebook (Meta) para la construcción de interfaces de usuario complejas, dinámicas y altamente reutilizables.

Su principal objetivo consiste en facilitar el desarrollo de aplicaciones capaces de actualizar la interfaz de forma eficiente cuando cambian los datos.

A diferencia de otros frameworks completos, React se centra exclusivamente en la capa de presentación de la aplicación.

En una arquitectura moderna, React suele ocupar la posición:

Usuario
↓
Interfaz React
↓
Servicios
↓
APIs
↓
Base de datos

En nuestro nuevo proyecto:

Ionic React
↓
Supabase
↓
PostgreSQL

2. Filosofía de React

React introduce una idea fundamental:

La interfaz es una representación del estado.

Esto significa que la pantalla no se modifica manualmente.

En lugar de ello:

Cambian los datos
↓
React detecta el cambio
↓
React actualiza la vista

El programador trabaja sobre los datos.

React se encarga de reflejar dichos cambios en la interfaz.

3. Características principales de React

Programación basada en componentes

Toda la aplicación se divide en pequeñas piezas independientes.

Por ejemplo:

App
│
├── Login
│
├── Clientes
│
├── FormularioCliente
│
└── Navbar

Cada componente posee:

- lógica
- presentación
- estado
- comportamiento

propios.

Reutilización

Un componente puede utilizarse múltiples veces.

Por ejemplo:

Botón

o

FormularioCliente

pueden aparecer en distintas páginas de la aplicación.

Esto reduce duplicidades y mejora el mantenimiento.

Programación declarativa

React utiliza un enfoque declarativo.

No se indica:

cómo modificar la pantalla

sino:

qué pantalla debe mostrarse

para un determinado estado de la aplicación.

Virtual DOM

React no modifica directamente el DOM del navegador.

Mantiene una representación ligera denominada:

Virtual DOM

Cuando cambian los datos:

Estado anterior
↓
Nuevo estado
↓
Comparación
↓
Actualización mínima necesaria

Este mecanismo mejora significativamente el rendimiento.

4. React dentro de Ionic

Ionic permite utilizar varios frameworks:

- Angular
-
- React
-
- Vue

En nuestro caso utilizaremos:

Ionic React

Arquitectura:

React
↓
Componentes Ionic
↓
Android
iOS
Web

5. React y TypeScript

Actualmente el ecosistema React utiliza principalmente TypeScript.

TypeScript aporta:

- tipado estático
- interfaces
- validación durante compilación
- autocompletado
- detección temprana de errores

Ejemplo conceptual:

Cliente
├── id
├── nombre
├── email
└── teléfono

TypeScript permite definir formalmente dicha estructura.

6. Concepto de Componente

El componente es la unidad básica de React.

Un componente es una función que devuelve una interfaz de usuario.

Arquitectónicamente:

Componente
↓
Recibe datos
↓
Procesa información
↓
Genera interfaz

En nuestro proyecto habrá componentes como:

- ClientsPage
- LoginPage
- ClientForm
- Navbar

7. JSX

React utiliza una sintaxis denominada JSX.

JSX combina:

JavaScript
+
HTML

permitiendo construir interfaces de forma declarativa.

Conceptualmente:

Datos
↓
JSX
↓
Interfaz

Aunque visualmente se parece a HTML, internamente es código JavaScript.

8. Estado (State)

Uno de los conceptos más importantes de React.

El estado representa información dinámica de un componente.

Ejemplos:

Usuario autenticado

Lista de clientes

Texto de búsqueda

Formulario de edición

Cuando el estado cambia:

React vuelve a renderizar
el componente

mostrando la nueva información.

9. Renderizado

Renderizar significa:

Transformar datos
en elementos visuales

Proceso:

Datos
↓
Componente
↓
React
↓
HTML generado

Cada vez que cambian los datos:

Nuevo render

10. Hooks

Los Hooks son funciones especiales introducidas para dotar de comportamiento a los componentes.

Los más importantes son:

useState

Permite gestionar estado local.

Se utilizará para:

Clientes

Login

Formularios

Mensajes

useEffect

Permite ejecutar lógica asociada al ciclo de vida del componente.

Se utilizará para:

Consultar Supabase

Cargar clientes

Refrescar información

useNavigate

Permite realizar navegación programática.

Equivalente conceptual a:

Router.navigate

de Angular.

11. Flujo de datos en React

React sigue un flujo de datos unidireccional.

Arquitectura:

Estado
↓
Componente Padre
↓
Props
↓
Componente Hijo

12. Props

Las Props representan información enviada de un componente a otro.

Funcionan como parámetros de entrada.

Permiten:

Reutilización

Comunicación

Configuración

entre componentes.

13. React Router

Las aplicaciones React modernas utilizan React Router para gestionar la navegación.

Permite definir:

/login

/clients

/new-client

/edit-client/:id

De forma similar a Angular Router.

14. Gestión de Servicios

En Angular utilizábamos:

ClientsService

Con React seguiremos una filosofía similar.

Arquitectura recomendada:

Pages
↓
Services
↓
Supabase

Evitando acceder directamente a Supabase desde todos los componentes.

15. React e Ionic

Ionic proporciona componentes adaptados a React.

Por ejemplo:

IonPage

IonHeader

IonContent

IonButton

IonInput

IonList

IonItem

Estos componentes mantienen la apariencia nativa en:

Android

iOS

Web

16. Relación entre React y Supabase

En nuestro proyecto la arquitectura será:

React Component
↓
ClientsService
↓
Supabase SDK
↓
Supabase Auth
↓
PostgreSQL

Las operaciones CRUD seguirán siendo:

Create
Read
Update
Delete

pero ahora implementadas mediante React en lugar de Angular.

17. Diferencias principales entre Angular y React

Angular

Framework completo.

Incluye:

- Router
- Inyección de dependencias
- Formularios
- HttpClient
- Guards

integrados.

React

Biblioteca más ligera.

Permite elegir:

- Router
- Gestión de estado
- Librerías auxiliares
- Arquitectura

según las necesidades del proyecto.

Angular

Más estructurado.

Más normativo.

Menos flexible.

React

Más flexible.

Más modular.

Mayor libertad arquitectónica.

18. Arquitectura objetivo del proyecto

El nuevo proyecto AppClients React se construirá siguiendo la siguiente estructura:

src

├── pages

│ ├── ClientsPage

│ ├── LoginPage

│ ├── NewClientPage

│ └── EditClientPage

├── services

│ ├── ClientsService

│ └── AuthService

├── interfaces

│ └── Client

├── config

│ └── supabase

├── routes

└── guards

React es una biblioteca basada en componentes, orientada al desarrollo de interfaces dinámicas mediante programación declarativa, gestión eficiente del estado y renderizado optimizado mediante Virtual DOM.

Combinada con Ionic y Supabase permite construir aplicaciones modernas multiplataforma capaces de ejecutarse en navegador, Android e iOS utilizando una única base de código y una arquitectura muy próxima a la utilizada actualmente en proyectos empresariales reales.

¿Por qué utilizar React en lugar de Angular en un proyecto Ionic?

Cuando se desarrolla una aplicación móvil híbrida con Ionic, una de las primeras decisiones arquitectónicas consiste en seleccionar el framework Frontend que servirá de base para el proyecto.

Ionic permite trabajar principalmente con:

- Angular
- React
- Vue

Históricamente, Ionic nació muy ligado a Angular, pero en los últimos años React ha experimentado un crecimiento extraordinario y se ha convertido en una de las opciones más utilizadas para el desarrollo de aplicaciones web y móviles.

La elección entre Angular y React no implica que una tecnología sea mejor que la otra de forma absoluta. La decisión dependerá de factores como la complejidad del proyecto, el tamaño del equipo, la experiencia previa de los desarrolladores y las necesidades de mantenimiento futuras.

Diferencia conceptual fundamental

Angular

Angular es un framework completo.

Proporciona de forma integrada:

- Routing
- Formularios
- Inyección de dependencias
- Validación
- Guards
- HttpClient
- Gestión de estados básica
- Testing

Angular define prácticamente toda la arquitectura de la aplicación.

React

React es una biblioteca especializada en la construcción de interfaces de usuario.

React proporciona:

- Componentes
- Estado
- Renderizado
- Hooks

El resto de funcionalidades pueden elegirse según las necesidades del proyecto.

Esto proporciona una flexibilidad considerablemente mayor.

Ventaja 1. Menor curva de aprendizaje inicial

Uno de los aspectos más valorados por los desarrolladores que comienzan con Ionic es la simplicidad de React.

Un desarrollador con conocimientos de:

- HTML
- CSS
- JavaScript

puede empezar a construir interfaces funcionales relativamente rápido.

Angular requiere asimilar conceptos adicionales como:

- Decoradores
- Módulos
- Inyección de dependencias
- Servicios
- RxJS
- Ciclo de vida específico

Por tanto:

React suele resultar más accesible para equipos pequeños y proyectos académicos.

Ventaja 2. Menor cantidad de código

En muchos escenarios React permite implementar la misma funcionalidad utilizando menos código.

Por ejemplo:

Formulario
Listado
Navegación

suelen requerir menos estructura inicial.

Esto reduce:

- tiempo de desarrollo
- complejidad
- mantenimiento

especialmente en aplicaciones pequeñas o medianas.

Ventaja 3. Más cercano a JavaScript puro

React extiende JavaScript utilizando JSX.

La filosofía es:

JavaScript genera la interfaz

Angular introduce su propia estructura:

Templates
Decoradores
Servicios
Módulos
Directivas

React suele resultar más natural para desarrolladores procedentes del ecosistema JavaScript tradicional.

Ventaja 4. Ecosistema extremadamente amplio

Actualmente React posee uno de los ecosistemas más grandes del desarrollo Frontend.

Existen miles de librerías para:

- autenticación
- formularios
- gráficos
- mapas
- estados globales
- testing
- inteligencia artificial

Esto facilita enormemente la integración de funcionalidades avanzadas.

Ventaja 5. Alto rendimiento

React utiliza Virtual DOM para optimizar las actualizaciones de interfaz.

Cuando cambian los datos:

Estado anterior
↓
Comparación
↓
Actualización mínima

Este enfoque reduce operaciones innecesarias sobre el DOM real y permite construir interfaces muy fluidas.

Ventaja 6. Excelente integración con Supabase

La arquitectura React encaja muy bien con Supabase debido a su simplicidad.

La estructura suele ser:

Page
↓
Service
↓
Supabase

Las operaciones CRUD pueden implementarse de forma directa y muy intuitiva.

Esto resulta especialmente útil en proyectos donde Supabase sustituye completamente al backend tradicional.

Ventaja 7. Muy demandado en el mercado laboral

React se ha convertido en una de las tecnologías más demandadas en el desarrollo Frontend.

Muchas empresas utilizan:

- React Web
- React Native
- Next.js
- Ionic React

Por tanto, aprender React proporciona competencias muy transferibles a distintos tipos de proyectos.

Ventaja 8. Mejor adaptación a proyectos pequeños y medianos

En proyectos tales como:

- gestión de clientes
- inventarios
- reservas
- aplicaciones educativas
- paneles administrativos

React suele permitir una mayor velocidad de desarrollo inicial.

La cantidad de estructura obligatoria es menor que en Angular.

¿Qué ventajas mantiene Angular?

Arquitectura más estricta

Angular obliga a seguir una estructura muy definida:

Component
Service
Module
Guard
Routing

Esto resulta especialmente útil en:

- equipos grandes
- proyectos corporativos
- aplicaciones de larga duración

Inyección de dependencias integrada

Angular incorpora un sistema de Dependency Injection extremadamente robusto.

Esto facilita:

- testing
- mantenimiento
- reutilización

sin necesidad de herramientas adicionales.

Framework completo

Angular proporciona prácticamente todas las herramientas necesarias desde el primer momento.

No es necesario elegir librerías para cada funcionalidad.

Mayor homogeneidad

Dos proyectos Angular suelen parecerse bastante entre sí.

En React es posible encontrar arquitecturas muy diferentes para resolver el mismo problema.

React en AppClients

Para nuestro proyecto AppClients basado en:

Ionic
React
Supabase
PostgreSQL

React aporta varias ventajas especialmente relevantes.

Desarrollo más rápido

La arquitectura CRUD puede implementarse de forma muy directa.

Menos complejidad inicial

No necesitamos:

Módulos
Decoradores
Inyección compleja

como ocurre en Angular.

Integración sencilla con Supabase

Las consultas, autenticación y operaciones CRUD pueden realizarse mediante Hooks y servicios muy ligeros.

Menor cantidad de ficheros

La estructura suele ser más compacta.

AppClients React

Ionic React + Supabase + PostgreSQL + Bootstrap

Desarrollo completo de un CRUD sobre la tabla `clients`

1. Objetivos de la práctica

En esta práctica vamos a construir una aplicación completa utilizando:

- Ionic React
- React Router
- TypeScript
- Supabase
- PostgreSQL
- Bootstrap
- Git
- GitHub
- Vercel

La aplicación permitirá:

Listar clientes
Crear clientes
Modificar clientes
Eliminar clientes

utilizando la tabla:

clients

existente en Supabase.

Arquitectura final

React Components

↓

Services

↓

Supabase SDK

↓

Supabase API

↓

PostgreSQL

2. Preparación de Supabase

Suponemos que ya existe la tabla.

Tabla clients

```
create table clients (

    id bigint generated always as identity primary key,

    name varchar(100),

    email varchar(100),

    phone varchar(20)

);
```

Datos iniciales

```
insert into clients
(name,email,phone)

values

('Juan',
'juan@test.com',
'600111111'),

('Ana',
'ana@test.com',
'600222222');
```

3. Crear proyecto Ionic React

Abrir terminal:

ionic start appclients-react blank
--type=react

Entrar al proyecto:

cd appclients-react

Ejecutar:

ionic serve

Comprobar:

http://localhost:8100

4. Instalar Supabase

npm install @supabase/supabase-js

5. Instalar Bootstrap

npm install bootstrap

Configurar Bootstrap

Archivo:

src/index.tsx

Añadir:

import 'bootstrap/dist/css/bootstrap.min.css';

6. Estructura del proyecto

Crear la siguiente estructura:

src

├── config

│ └── supabase.ts

├── services

│ └── ClientsService.ts

├── interfaces

│ └── Client.ts

├── pages

│ ├── ClientsPage.tsx

│ ├── NewClientPage.tsx

│ └── EditClientPage.tsx

├── App.tsx

└── ...

7. Variables de entorno

Crear:

.env

Contenido:

VITE_SUPABASE_URL=https://xxxx.supabase.co

VITE_SUPABASE_KEY=xxxxxxxxxxx

Añadir al .gitignore .env

8. Configuración Supabase

src/config/supabase.ts

```
import { createClient }
from '@supabase/supabase-js';

const supabaseUrl =
  import.meta.env
        .VITE_SUPABASE_URL;

const supabaseKey =
  import.meta.env
        .VITE_SUPABASE_KEY;

export const supabase =
  createClient(
        supabaseUrl,
        supabaseKey
  );
```

React con Vite expone las variables de entorno mediante:

import.meta.env

9. Crear interfaz

interfaces/Client.ts

```
export interface Client {

  id?: number;

  name: string;

  email: string;

  phone: string;

}
```

Ventaja

Mejor tipado.

Mayor seguridad.

Autocompletado.

10. Crear Service

services/ClientsService.ts

```
import { supabase }
from '../config/supabase';

import { Client }
from '../interfaces/Client';

export class ClientsService {

  static async getClients() {

        const { data, error } =

        await supabase
        .from('clients')
        .select('');

        if(error) {

        throw error;

        }

        return data;

  }

  static async getClient(
        id:number
  ) {

        const { data, error } =

        await supabase

        .from('clients')
```

```
        .select('*')

        .eq('id',id)

        .single();

        if(error) {

        throw error;

        }

        return data;

}

static async addClient(
        client:Client
) {

        const { error } =

        await supabase

        .from('clients')

        .insert([client]);

        if(error) {

        throw error;

        }

}

static async updateClient(
        id:number,
        client:Client
```

```
) {
        const { error } =
        await supabase
        .from('clients')
        .update(client)
        .eq('id',id);
        if(error) {
        throw error;
        }
}
static async deleteClient(
        id:number
) {
        const { error } =
        await supabase
        .from('clients')
        .delete()
        .eq('id',id);
        if(error) {
        throw error;
        }
```

```
  }

}
```

11. Configurar Routing

App.tsx

Importar:

```
import {
  BrowserRouter,
  Routes,
  Route,
  Navigate
} from 'react-router-dom';
```

Configurar:

```
<BrowserRouter>

  <Routes>

        <Route

        path="/"

        element={
        <Navigate
        to="/clients"
        />
        }

        />

        <Route

        path="/clients"

        element={
        <ClientsPage/>
        }

        />

        <Route

        path="/new-client"

        element={
        <NewClientPage/>
        }

        />

        <Route

        path="/edit-client/:id"
```

```
        element={
        <EditClientPage/>
        }

        />

  </Routes>

</BrowserRouter>
```

12. Página listado

ClientsPage.tsx

```
import {

  useEffect,

  useState

}
from 'react';

import {
  Link
}
from 'react-router-dom';

import {
  ClientsService
}
from '../services/ClientsService';

export default function ClientsPage() {

  const [

        clients,

        setClients

  ] = useState<any[]>([]);

  const loadClients =
        async () => {

        const data =

        await ClientsService
```

```
        .getClients();

        setClients(data);

        };

useEffect(() => {

        loadClients();

}, []);

return (

        <div className="container mt-4">

        <h2>

        Clientes

        </h2>

        <Link

        to="/new-client"

        className=
        "btn btn-primary mb-3">

        Nuevo Cliente

        </Link>

        <table
        className=
        "table table-bordered">

        <thead>

        <tr>
```

```
<th>ID</th>

<th>Nombre</th>

<th>Email</th>

<th>Teléfono</th>

<th>Acciones</th>

</tr>

</thead>

<tbody>

{clients.map(
client => (

<tr
key={client.id}>

<td>
{client.id}
</td>

<td>
{client.name}
</td>

<td>
{client.email}
</td>

<td>
{client.phone}
</td>

<td>
```

```
            ...

            </td>

            </tr>

            ))}

            </tbody>

            </table>

            </div>

    );

}
```

Explicación técnica

`useState`

Permite almacenar estado.

`useEffect`

Ejecuta lógica al cargar la página.

`setClients`

Actualiza el estado y obliga a React a renderizar nuevamente.

13. Alta de clientes

NewClientPage.tsx

```
import {
  useState
}
from 'react';

import {
  useNavigate
}
from 'react-router-dom';

import {
  ClientsService
}
from '../services/ClientsService';

export default function NewClientPage() {

  const navigate =
          useNavigate();

  const [

          client,

          setClient

  ] = useState({

          name:'',

          email:'',

          phone:''

  });
```

```
const save =
        async () => {

        await ClientsService
        .addClient(client);

        navigate(
        '/clients'
        );

        };

return (

        <div className="container mt-4">

        <h2>

        Nuevo Cliente

        </h2>

        <input

        className=
        "form-control mb-2"

        placeholder="Nombre"

        onChange={(e)=>

        setClient({

        ...client,

        name:
        e.target.value

        })
```

```
}

/>

<input

className=
"form-control mb-2"

placeholder="Email"

onChange={(e)=>

setClient({

...client,

email:
e.target.value

})

}

/>

<input

className=
"form-control mb-2"

placeholder="Teléfono"

onChange={(e)=>

setClient({

...client,
```

```
            phone:
            e.target.value

            })

            }

            />

            <button

            className=
            "btn btn-success"

            onClick={save}

            >

            Guardar

            </button>

            </div>

    );

}
```

14. Modificación

EditClientPage.tsx

Obtención del parámetro:

```
const { id } =
  useParams();
```

Carga inicial:

```
useEffect(() => {

  loadClient();

}, []);
```

Consultar:

```
const data =

  await ClientsService
          .getClient(
          Number(id)
          );
```

Actualizar:

```
await ClientsService
  .updateClient(

          Number(id),

          client

  );
```

Redirección:

```
navigate('/clients');
```

15. Eliminar

En la tabla.

Botón:

```
<button

className=
"btn btn-danger"

onClick={()=>

  remove(
        client.id
  )

}

>

Eliminar

</button>
```

Método:

```
const remove =
async (
 id:number
) => {

 const ok =

        confirm(
        '¿Eliminar cliente?'
        );

 if(!ok) return;
```

```
  await ClientsService
          .deleteClient(id);

  loadClients();
};
```

Flujo Delete

Botón
  ↓
Confirmación
  ↓
Supabase
  ↓
DELETE
  ↓
Refresco tabla

16. Pruebas

Comprobar:

SELECT

Listado.

INSERT

Nuevo cliente.

UPDATE

Modificar cliente.

DELETE

Eliminar cliente.

---

17. Git

Inicializar:

git init

Crear:

.gitignore

node_modules

dist

.env

Guardar:

git add .

git commit -m "CRUD React Supabase"

18. GitHub

git remote add origin URL

git push -u origin main

19. Deploy en Vercel

Entrar en:

https://vercel.com

Importar repositorio.

Añadir variables:

VITE_SUPABASE_URL

VITE_SUPABASE_KEY

en:

Project Settings
→ Environment Variables

Deploy.

Resultado final

Obtendremos una aplicación Ionic React conectada a Supabase capaz de realizar operaciones CRUD completas sobre la tabla `clients`, utilizando React Router para navegación, Bootstrap para presentación visual, Git para control de versiones, GitHub para alojamiento del código y Vercel para publicación en producción.

AppClients React

Autenticación con Supabase

Solo usuarios autenticados pueden insertar clientes

Objetivo

Hasta este momento la aplicación permite:

- Consultar clientes
- Insertar clientes
- Modificar clientes
- Eliminar clientes

Sin embargo, cualquier usuario que acceda a la aplicación puede crear registros.

Vamos a implementar una primera capa de seguridad donde:

Usuarios anónimos

↓

Pueden consultar clientes

Usuarios autenticados

↓

Pueden insertar clientes

Más adelante podremos extender esta misma arquitectura para proteger:

- Editar
- Eliminar
- Administración

1. Arquitectura de autenticación

La autenticación se realizará mediante:

React
↓
AuthService
↓
Supabase Auth
↓
JWT
↓
Session
`

Una vez autenticado el usuario:

Supabase genera:

Access Token
Refresh Token
Session

React únicamente consulta el estado de la sesión.

2. Activar Authentication en Supabase

Entrar en:

Supabase
→ Authentication
→ Users

Crear un usuario:

profesor@centro.es

123456

o:

alumno@centro.es

123456

3. Crear AuthService

Estructura

src

└── services

└── AuthService.ts

AuthService.ts

```
import { supabase }
from '../config/supabase';

export class AuthService {

  static async login(
        email:string,
        password:string
  ) {

        const {

        data,

        error

        } = await supabase.auth

        .signInWithPassword({

        email,

        password

        });

        if(error) {
```

```
            throw error;
            }
            return data;
    }
    static async logout() {
            await supabase.auth.signOut();
    }
    static async getSession() {
            const {
            data
            } = await supabase.auth
            .getSession();
            return data.session;
    }
    static async isAuthenticated() {
            const session =
            await this.getSession();
            return !!session;
    }
}
```

Análisis técnico

login()

Realiza la autenticación mediante correo y contraseña.

getSession()

Pregunta a Supabase si existe una sesión activa.

isAuthenticated()

Devuelve:

true

si existe una sesión válida.

Devuelve:

false

si el usuario no está autenticado.

4. Crear LoginPage

Estructura

pages

└── LoginPage.tsx

LoginPage.tsx

```
import {

  useState

} from 'react';

import {

  useNavigate

} from 'react-router-dom';

import {

  AuthService

} from '../services/AuthService';

export default function LoginPage() {

  const navigate =
        useNavigate();

  const [

        email,

        setEmail
```

```
] = useState('');

const [

        password,

        setPassword

] = useState('');

const login =
        async () => {

        try {

        await AuthService.login(

        email,

        password

        );

        navigate('/clients');

        }
        catch {

        alert(
        'Credenciales incorrectas'
        );

        }

        };

return (

        <div className="container mt-4">
```

```
<h2>

Login

</h2>

<input

className="form-control mb-2"

type="email"

placeholder="Email"

value={email}

onChange={e =>
setEmail(
e.target.value
)
}

/>

<input

className="form-control mb-3"

type="password"

placeholder="Contraseña"

value={password}

onChange={e =>
setPassword(
e.target.value
)
}
```

```
            />
            <button
            className="btn btn-primary"
            onClick={login}
            >
            Entrar
            </button>
            </div>
    );
}
```

5. Añadir ruta Login

App.tsx

```
<Route

  path="/login"

  element={
          <LoginPage/>
  }

/>
```

6. Comprobar autenticación

Vamos a necesitar conocer el estado de la sesión desde la página de clientes.

ClientsPage.tsx

Crear:

```
const [

  isLogged,

  setIsLogged

] = useState(false);
```

7. Cargar estado de autenticación

Dentro de:

useEffect()

Añadimos:

```
const checkAuth =
  async () => {

        const logged =

        await AuthService
        .isAuthenticated();

        setIsLogged(
        logged
        );

  };
```

Y ejecutamos:

```
checkAuth();
```

Resultado

Ahora disponemos de:

isLogged

que indica si existe sesión.

8. Ocultar botón Nuevo Cliente

Actualmente:

```
<Link

  to="/new-client"

  className=
  "btn btn-primary"

>

  Nuevo Cliente

</Link>
```

Cambiar por:

```
{

  isLogged &&

  <Link

        to="/new-client"

        className=
        "btn btn-primary"

  >

        Nuevo Cliente

  </Link>

}
```

¿Qué hace?

Si:

isLogged = true

aparece el botón.

Si:

isLogged = false

desaparece.

9. Proteger la página NewClient

Ocultar el botón no es suficiente.

Un usuario podría escribir:

/new-client

manualmente.

Necesitamos proteger la ruta.

10. Crear ProtectedRoute

src/routes/ProtectedRoute.tsx

```
import {
  Navigate
} from 'react-router-dom';
import {
  useEffect,
  useState
} from 'react';
import {
  AuthService
} from '../services/AuthService';
export default function ProtectedRoute({
  children
}:any) {
  const [
        allowed,
        setAllowed
  ] = useState(false);
  const [
```

```
        loading,

        setLoading

] = useState(true);

useEffect(() => {

        const checkAuth =
        async () => {

        const logged =

        await AuthService
        .isAuthenticated();

        setAllowed(
        logged
        );

        setLoading(
        false
        );

        };

        checkAuth();

}, []);

if(loading) {

        return <p>Cargando...</p>;

}

if(!allowed) {

        return (
```

```
        <Navigate
        to="/login"
        />
        );

  }

  return children;

}
```

Concepto técnico

Un ProtectedRoute es el equivalente conceptual a un Guard de Angular.

Su responsabilidad consiste en decidir:

¿Puede cargarse esta página?

antes de renderizarla.

11. Aplicar protección

App.tsx

Actualmente:

```
<Route

  path="/new-client"

  element={
        <NewClientPage/>
  }

/>
```

Modificar:

```
<Route

  path="/new-client"

  element={

        <ProtectedRoute>

        <NewClientPage/>

        </ProtectedRoute>

  }

/>
```

Flujo de ejecución

Usuario

↓

/new-client

↓

ProtectedRoute

↓

isAuthenticated()

↓

TRUE

↓

NewClientPage

o:

Usuario

↓

/new-client

↓

ProtectedRoute

↓

FALSE

↓

Redirect /login

12. Añadir Logout

ClientsPage.tsx

```
const logout =
  async () => {

        await AuthService
        .logout();

        window.location.href =
        '/login';

  };
```

Botón

```
{

  isLogged &&

  <button

        className=
        "btn btn-secondary"

        onClick={logout}

  >

        Logout

  </button>

}
```

13. Arquitectura final

LoginPage

↓

AuthService

↓

Supabase Auth

↓

JWT

↓

Session

↓

ProtectedRoute

↓

NewClientPage

↓

ClientsService

↓

Supabase

↓

PostgreSQL

Resultado final

Un usuario no autenticado:

Puede consultar clientes

No puede acceder a Nuevo Cliente

No puede insertar clientes

Un usuario autenticado:

Puede consultar clientes

Puede acceder al formulario

Puede insertar clientes